

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – SAST SIMULATION

Course Code:	CSA5803	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO3 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1
Weightage:	30% of CIA	Total Marks:	100
CO3 Statement:	Simulate Static Application Security Testing (SAST) using synthetic data		
Student Name:	Syed Saniya Iqbal	Reg. No.:	192465043
Section / Batch:	CSE-Cybersecurity	Date:	14/8/26

Instructions to Students

- Ensure all diagrams and workflow illustrations are **original, clear, and professionally formatted**.
- Include **all editable source files** (such as .yaml, .py, requirements.txt, .pre-commit-config.yaml, and any diagram source files if applicable).
- Submit **both** the **GitHub repository link** and the **final PDF report** through **Google Classroom** before the specified deadline.
- Verify that the GitHub repository is accessible and contains all required project files, screenshots, and documentation.

Question Type	Focus Area	Questions	Marks
SAST SIMULATION	Creative Things and concept grasping	Q1	Q1 –100
GRAND TOTAL:		100 Marks	

Code of Conduct

*I **Syed Saniya Iqbal(192465043)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **Syed Saniya Iqbal(192465043)**.

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

SAST SIMULATION USING A REAL-LIFE SCENARIO

1. Title

Static Application Security Testing (SAST) Simulation for an Online Banking Application Using Synthetic Data

2. Real-Life Scenario

Scenario: Online Banking Web Application

Online banking applications are widely used by customers to:

- Log in securely
- Check account balances
- Transfer money
- Pay bills
- View transaction history
- Manage user profiles

For this SAST simulation, a **synthetic online banking web application** is considered.

The application does **not** connect to any real bank, database, or payment system. All usernames, transactions, account numbers, source-code samples, vulnerabilities, and security results are artificially generated.

Main Features

No.	Module
1	User Login
2	Account Dashboard
3	Balance Checking
4	Money Transfer
5	Transaction History
6	User Profile
7	Logout

The objective is to simulate how a security team can identify application security weaknesses **before the application is deployed into production**.

3. Problem Statement

A banking application handles sensitive information such as usernames, passwords, account details, and financial transactions. Security vulnerabilities could allow attackers to access unauthorized information or manipulate transactions.

Therefore, **Static Application Security Testing (SAST)** is performed on application source code to identify potential security weaknesses without executing the application.

The simulation will:

- Generate synthetic source-code samples.
- Introduce intentional security vulnerabilities.
- Scan source code using simulated SAST rules.
- Detect vulnerabilities.

- Classify vulnerabilities according to severity.
- Calculate a security risk score.
- Display results using tables and charts.
- Provide recommended remediation actions.

4. Objectives

Main Objective

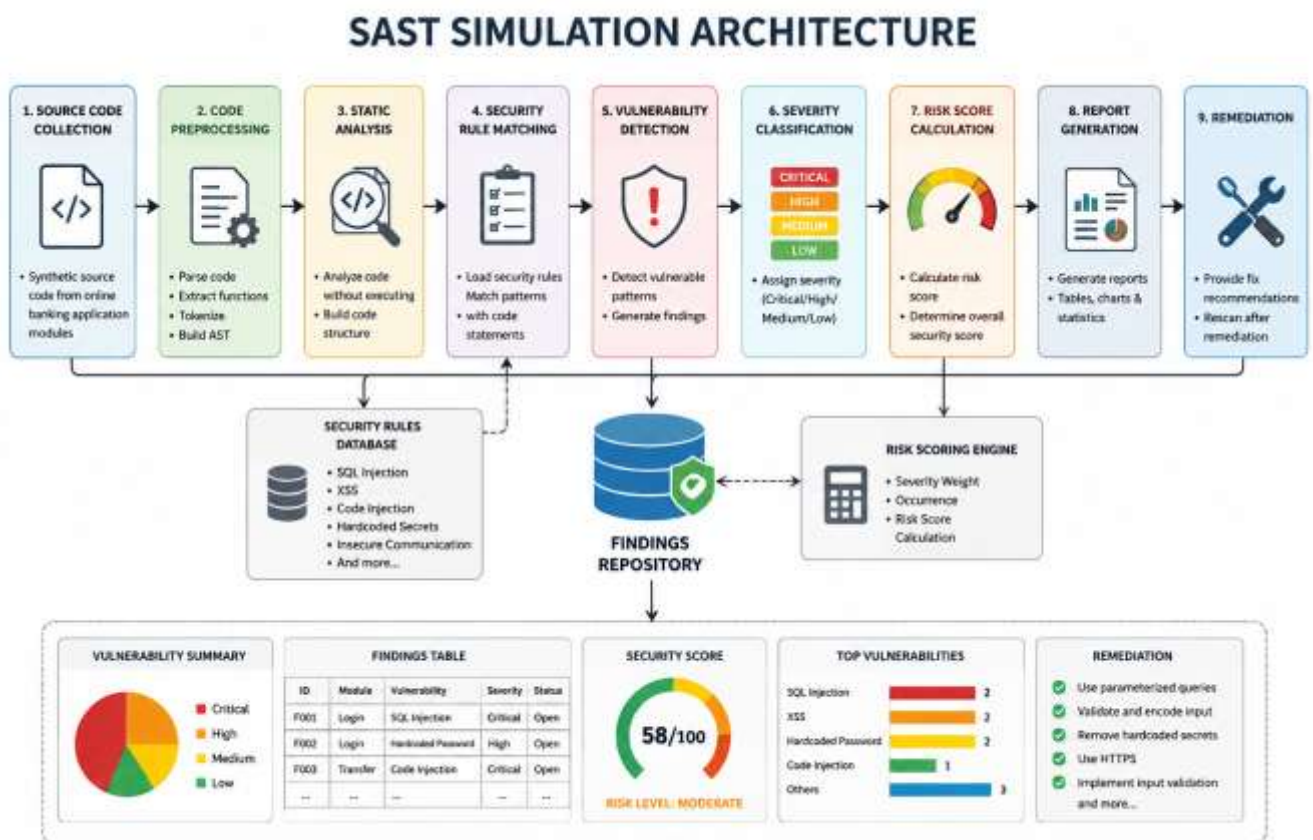
To simulate a SAST process for an online banking application using synthetic data.

Specific Objectives

1. Understand the working principle of SAST.
2. Identify security vulnerabilities in source code.
3. Simulate automated source-code scanning.
4. Classify vulnerabilities as Critical, High, Medium, or Low.
5. Calculate an overall security score.
6. Recommend suitable remediation techniques.
7. Visualize security scanning results.

5. SAST Simulation Workflow

Architecture



Workflow

Source Code → Preprocessing → Static Analysis → Rule Matching → Vulnerability Detection → Severity Classification → Risk Calculation → Report Generation → Remediation

6. Synthetic Source-Code Dataset

The simulation uses artificial code snippets instead of real banking source code.

ID	Module	Synthetic Code Pattern	Vulnerability
S001	Login	query = "SELECT * FROM users WHERE user='" + username + '";"	SQL Injection
S002	Login	password = "admin123"	Hardcoded Password
S003	Transfer	eval(userInput)	Code Injection
S004	Profile	element.innerHTML = userName	XSS
S005	Transfer	http://bank.local/transfer	Insecure HTTP
S006	Login	console.log(password)	Sensitive Data Exposure
S007	Database	SELECT * FROM accounts	Excessive Data Exposure
S008	Session	sessionTimeout = 3600000	Weak Session Configuration
S009	Transfer	amount = request.amount	Missing Input Validation
S010	Login	token = "BANK_TOKEN_123"	Hardcoded Secret

Note: The dataset is completely synthetic and is used only for educational simulation.

7. Security Rules Used in the Simulation

Rule	Vulnerability	Pattern	Severity	Recommendation
R1	SQL Injection	SQL query + direct user input	Critical	Use parameterized queries
R2	Hardcoded Password	password = "somePassword"	High	Use environment variables/secrets manager
R3	Cross-Site Scripting	innerHTML = userInput	High	Validate and encode user input
R4	Code Injection	eval(userInput)	Critical	Avoid dynamic code execution
R5	Insecure Communication	http://	High	Use HTTPS
R6	Sensitive Data Exposure	console.log(password)	High	Never log passwords/tokens
R7	Missing Input Validation	amount = request.amount	Medium	Validate type, range and format
R8	Hardcoded Secret	token = "BANK_TOKEN_123"	High	Store secrets outside source code

8. Simulation Steps

Step 1 – Define the Application

Create a conceptual online banking application containing:

- Login
- Dashboard
- Transfer
- Profile
- Transaction modules

The application does not need to connect to a real banking system.

Step 2 – Generate Synthetic Source Code

Example:

```
let username = input.username;
```

```
let query =  
"SELECT * FROM users WHERE username='" + username + "'";
```

The simulator identifies the direct combination of a database query and user-controlled input.

Step 3 – Create SAST Rules

Example:

```
IF source contains "SELECT"  
AND source contains user input  
THEN  
    Report SQL Injection
```

Another rule:

```
IF source contains "innerHTML"  
AND source contains user input  
THEN  
    Report Cross-Site Scripting
```

Step 4 – Scan the Synthetic Code

The simulator scans each source-code snippet line by line.

Example:

```
S001 → SQL Injection detected  
S002 → Hardcoded Password detected  
S003 → Code Injection detected
```

Step 5 – Generate Findings

Finding ID	Module	Vulnerability	Severity	Line
F001	Login	SQL Injection	Critical	12
F002	Login	Hardcoded Password	High	18
F003	Transfer	Code Injection	Critical	34
F004	Profile	XSS	High	41
F005	Transfer	Insecure HTTP	High	48

9. Severity Scoring

Severity	Score
Critical	10
High	7
Medium	4
Low	1

Examples

- SQL Injection → Critical
- Code Injection → Critical
- Hardcoded Password → High
- XSS → High
- Missing Input Validation → Medium

10. Risk Score Calculation

Formula

Risk Score = Severity Score × Occurrence

Example:

SQL Injection has a severity score of 10 and occurs twice.

$$\begin{aligned}\text{Risk Score} &= 10 \times 2 \\ &= 20\end{aligned}$$

Total Risk

Total Risk = Σ Individual Risk Scores

Example:

$$\begin{aligned}\text{Maximum Possible Risk} &= 100 \\ \text{Detected Risk} &= 42\end{aligned}$$

$$\begin{aligned}\text{Security Score} &= 100 - 42 \\ &= 58/100\end{aligned}$$

Final Security Score

Application Security Score = 58%

11. Risk Classification

Security Score	Security Status
90–100	Excellent
75–89	Good
50–74	Moderate
25–49	Poor
0–24	Critical

Sample Result

Security Score = 58%

Security Status = MODERATE RISK

12. Sample SAST Scan Result

Severity	Number of Findings
----------	--------------------

Critical	2
High	5
Medium	2
Low	1
Total	10

Result Summary

Total Files Scanned : 5
Total Lines Scanned : 250
Total Vulnerabilities : 10

Critical Vulnerabilities : 2
High Vulnerabilities : 5
Medium Vulnerabilities : 2
Low Vulnerabilities : 1

Security Score : 58%
Risk Level : MODERATE

13. Example Vulnerability Reports

Finding F001 – SQL Injection

Module: Login
Severity: Critical

Vulnerable Code

```
query = "SELECT * FROM users WHERE username='" + username + "'";
```

Reason

User input is directly concatenated into a database query.

Potential Impact

An attacker could manipulate the database query and potentially access or modify unauthorized information.

Recommended Fix

Use parameterized queries.

Finding F002 – Hardcoded Password

Module: Login
Severity: High

Vulnerable Code

```
password = "admin123";
```

Reason

Sensitive credentials are stored directly in the source code.

Recommended Fix

Use secure environment variables or a secrets management system.

Finding F003 – Cross-Site Scripting

Module: Profile
Severity: High

Vulnerable Code
element.innerHTML = userName;

Reason

Untrusted user input is inserted directly into HTML.

Recommended Fix

Validate and encode user input before displaying it.

14. Simulation of Remediation

Before Fix
query = "SELECT * FROM users WHERE username=" + username + "";

After Fix
query = "SELECT * FROM users WHERE username = ?";

The vulnerability status changes:

OPEN → FIXED

The security score is then recalculated after remediation.

15. Before and After Comparison

Parameter	Before Remediation	After Remediation
Critical Issues	2	0
High Issues	5	2
Medium Issues	2	1
Low Issues	1	1
Total Issues	10	4
Security Score	58%	86%
Risk Level	Moderate	Good

Improvement
Before Remediation → 58% → MODERATE
After Remediation → 86% → GOOD

This demonstrates the effectiveness of fixing vulnerabilities identified by SAST.

16. Proposed Simulation Interface Dashboard

Display:

SAST SECURITY DASHBOARD	
Files Scanned	: 5
Lines Scanned	: 250
Vulnerabilities	: 10
Critical Issues	: 2
High Issues	: 5
Medium Issues	: 2
Security Score	: 58/100
Risk Level	: MODERATE

Source Code Panel

Displays the synthetic source code being scanned.

Scan Controls

[Start Scan] [Pause]

[Next Finding] [Auto Scan]

[Reset]

Vulnerability Panel

Vulnerability : SQL Injection
Severity : Critical
Module : Login
Line : 12
Status : Open

Security Score

SECURITY SCORE

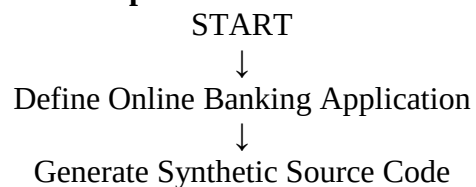
58 / 100

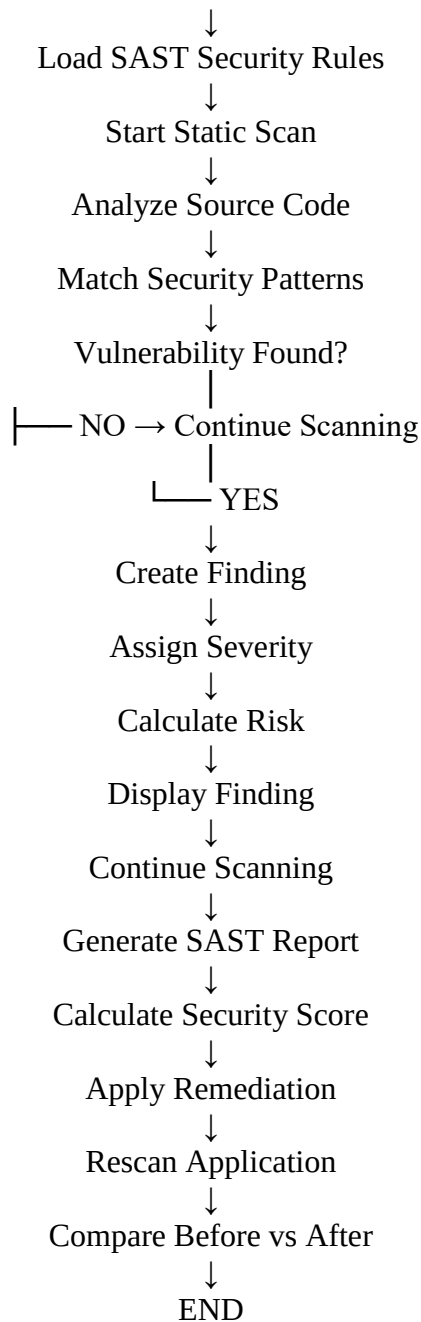
MODERATE RISK

Critical :  2
High :  5
Medium :  2
Low :  1

Results Chart

17. Complete Simulation Flow





18. Expected Outcome

The SAST simulation should successfully:

1. Scan synthetic source-code data.
2. Detect predefined security vulnerabilities.
3. Classify vulnerabilities according to severity.
4. Calculate application security risk.
5. Display vulnerabilities in a structured report.
6. Recommend remediation techniques.
7. Demonstrate improved security after fixing vulnerabilities.
8. Generate a final SAST security score.

19. Conclusion

This project demonstrates a **Static Application Security Testing (SAST) simulation for an online banking application using synthetic data.**

The simulation follows the complete security-testing lifecycle, beginning with source-code collection and ending with vulnerability remediation and rescan.

Using predefined security rules, the simulator identifies vulnerabilities such as:

- SQL Injection
- Cross-Site Scripting
- Code Injection
- Hardcoded Credentials
- Insecure Communication
- Sensitive Data Exposure
- Missing Input Validation

The before-and-after comparison demonstrates how remediation reduces application risk and improves the overall security score.

Since all source code, user information, account details, and transaction data are synthetic, the simulation safely demonstrates real-world SAST concepts without using actual banking data.

Final Result

The proposed SAST simulator successfully models the process of identifying, classifying, scoring, and remediating application security vulnerabilities using synthetic data.